

Trabajo Práctico 1:

Ordenamiento V1.0



06/09/24

Alumno: Tomas Goncalves
Rei

CB100 - Algoritmos y
Estructuras de Datos

Mail: tgoncalves@fi.uba.ar

Profesor: Gustavo Schmidt

Padron: 111405

Indice

<i>Introduccion.....</i>	<i>3</i>
<i>Lenguaje y Herramientas</i>	<i>3</i>
<i>Funcionalidad</i>	<i>3</i>
<i>Algoritmos de Ordenamiento</i>	<i>4</i>
<i>Cuestionario.....</i>	<i>5</i>
1. ¿Qué es un Debug?.....	5
2. ¿Qué es un “Breakpoint”?.....	5
3. ¿Qué es “Step Into”, “Step Over” y “Step Out”?	5
<i>Manual de Usuario</i>	<i>6</i>
<i>Manual de Programador</i>	<i>7</i>
<i>Conclusion.....</i>	<i>9</i>

Introduccion

En este trabajo se busca generar una pieza de software que simule el funcionamiento de la comparativa entre los metodos de ordenamiento, a traves del manejo de un archivo con numeros enteros, eligiendo el algoritmo a implementar.

Lenguaje y Herramientas

Se utilizara el lenguaje de programacion **Java**, con el IDE **Eclipse** para realizar dicho proyecto y compilarlo.

La eleccion de utilizar Java se basa en el simple y familiar uso del lenguaje para poder programar nuestro proyecto de una forma mas eficiente y con uno de los lenguajes mas populares dentro de la programacion.

Como entorno de desarrollo utilizaremos Eclipse, ya que es un IDE que contiene las herramientas que necesitamos para administrar nuestro proyecto y poder desarrollar el software, pudiendo editar codigo, compilarlo, debuggearlo, etc.

Funcionalidad

Al manejar el programa, nos encontraremos con la clase “PrincipalTP”, que sera la que contiene la funcion principal y con la que comenzara a correr el software. Esta clase nos mostrara por la consola del IDE un menu donde se mostrara al usuario los diferentes algoritmos de ordenamiento y como seleccionar cada uno. Luego de elegir una de las opciones, el programa leera el archivo de texto compuesto por enteros, guardara los valores, los ordenara dependiendo el metodo seleccionado y luego creara un nuevo archivo de texto con los numeros enteros ordenados. A si mismo, se mostrara por consola el tiempo en milisegundos que tardo el algoritmo en ordenar el archivo.

Esto puede realizarse la cantidad de veces que lo requiera el usuario e ingrese la opcion de finalizar el programa.

Algoritmos de Ordenamiento

➤ Método de Burbujeo (Bubble Sort):

Algoritmo sencillo, pero ineficiente para grandes conjuntos de datos.

Funcionamiento:

- Realiza pasadas al arreglo (vector) y compara pares de elementos adyacentes.
- Si el par está desordenado, es decir que el primero es más grande, los intercambia.
- Se repite el proceso hasta que esté ordenado. En cada pasada, el elemento más grande burbujea hasta el final del arreglo.

➤ QuickSort:

Algoritmo eficiente y popular utilizando el enfoque de división y conquista.

Funcionamiento:

- Se selecciona un elemento como pivote.
- Se reorganiza el arreglo de forma tal que los elementos menores que el pivote queden a la izquierda, y los mayores a la derecha.
- Se aplica el mismo proceso en cada sublista hasta que cada una esté ordenada.

➤ Radix Sort:

Algoritmo que clasifica los números basándose en sus dígitos.

Funcionamiento:

- Procesa los elementos en función de sus dígitos, comenzando generalmente por el menos significativo.
- Para cada valor del dígito, los elementos se agrupan en contenedores.
- Luego, pasa al siguiente dígito más significativo y repite el proceso hasta que todos los dígitos hayan sido procesados.

Cuestionario

1. ¿Qué es un Debug?

Un **Debug** o depuración es el proceso de identificar, rastrear y corregir errores o problemas en el código de un programa. Cuando depuras un programa, puedes ejecutarlo de manera controlada para observar cómo se comporta en tiempo real, inspeccionando variables, el flujo de ejecución, y verificando si la lógica es correcta.

2. ¿Qué es un “Breakpoint”?

Un **Breakpoint** o punto de interrupción es una herramienta que se utiliza en el proceso de depuración. Es una marca que colocas en una línea específica del código, indicando que la ejecución del programa debe detenerse cuando se alcanza esa línea durante la ejecución. Esto te permite inspeccionar el estado del programa en ese punto, examinar valores de variables, el flujo de control, y decidir si continuar con la ejecución o realizar cambios.

3. ¿Qué es “Step Into”, “Step Over” y “Step Out”?

Los siguientes son comandos para controlar la ejecución del programa durante la depuración:

- **Step Into:** Comando que permite entrar dentro de una función o método que se está llamando en la línea de código actual. Si estás en una línea que llama a otro método y usas "Step Into", el depurador saltará al primer paso dentro de ese método, permitiéndote seguir el flujo de ejecución línea por línea dentro de él.

- **Step Over:** Comando que ejecuta la línea actual de código, pero si esa línea contiene una llamada a otro método, no entra en ese método. En lugar de eso, ejecuta todo el método de una vez y se detiene en la siguiente línea después de la llamada al método.

- **Step Out:** Comando que se utiliza para salir de la función o método actual. Si has utilizado "Step Into" para entrar en un método y deseas salir de él rápidamente y regresar al lugar desde donde fue llamado, utilizas "Step Out". El programa ejecuta el resto del método actual y se detiene en la línea inmediatamente después de la llamada al método en la función original.

Manual de Usuario

➤ Descripción de Programa

Este programa permite al usuario ordenar un archivo de texto que contiene números enteros utilizando tres métodos de ordenamiento diferentes:

Burbujeo, QuickSort y RadixSort. El usuario puede elegir el método de ordenamiento a través de un menú interactivo, y el programa generará un nuevo archivo con los números ordenados según el método seleccionado.

➤ Requisitos de Sistema

- Java 8 o superior
- Editor o IDE de Java (Eclipse en este caso)

➤ Instrucciones de Uso

- Asegurarse que contiene el archivo “valores.txt”, luego compilar y ejecutar la clase “PrincipalTP”

- Se despliega un menú como el siguiente:

```
Hola, vamos a ordenar un archivo de 3 maneras distintas, los metodos son:  
1-Burbujeo  
2-QuickSort  
3-RadixSort  
4-Salir  
Ingrese un numero dependiendo el metodo a implementar|
```

El usuario deberá ingresar un número del 1 al 4 dependiendo la opción que desee

- Si elige un número del 1 al 3, el programa leerá el archivo de “valores.txt” y creará un nuevo archivo con los valores ordenados con el método seleccionado y mostrará por pantalla el tiempo que tardó en realizar el proceso

- Una vez creado, el programa volverá a preguntar si quiere ordenar el archivo de otra manera, hasta que se ingrese el número 4 y finalice la ejecución

Manual de Programador

➤ Descripción del Proyecto

Este programa ordena números enteros contenidos en un archivo de texto utilizando diferentes algoritmos de ordenamiento (Burbujeo, QuickSort, RadixSort) y guarda los resultados en archivos distintos.

➤ Lenguaje de Programación: Java

➤ IDE Recomendado: Eclipse

➤ Estructura del Proyecto

- Paquete Principal: "ar.uba.fi.cb100.tp1"

- Clases:

-- PrincipalTP: Contiene el "main" que ejecuta el programa y mantiene la interacción con el usuario. Recibe la opción de ordenamiento, y ejecuta el algoritmo correspondiente.

-- AdministradorDeTexto: Lee un archivo de texto y devuelve un arreglo con enteros para, luego de ordenarlos, volverlos a escribir en un nuevo archivo de texto.

-- OrdenamientoDeArchivo: Coordina la ejecución de los métodos de ordenamiento y crea los nuevos archivos ordenados, recibiendo como parámetros, el arreglo de enteros y un número correspondiente del método seleccionado.

-- MetodoDeBurbujeo: Implementación de algoritmo de ordenamiento por Burbujeo del arreglo de enteros recibido.

-- MetodoDeQuickSort: Implementación de algoritmo de ordenamiento por QuickSort del arreglo de enteros recibido.

-- MetodoDeRadixSort: Implementación de algoritmo de ordenamiento por RadixSort del arreglo de enteros recibido.

➤ Orden de Ejecución

Al ejecutar la clase principal se despliega el menú al usuario donde podrá visualizar las opciones para elegir el método de ordenamiento que desea implementar en el archivo de “valores.txt”. Este mismo se lee utilizando el método

“AdministradorDeTexto.leerLineasDeTexto”.

Dependiendo el número de entrada del usuario, se llama a “OrdenamientoDeArchivo.crearArchivoOrdenado” con un método correspondiente y su arreglo de valores. Luego este arreglo ordenado se guarda en un nuevo archivo de salida específico para cada algoritmo.

Al finalizar, se da una nueva oportunidad al usuario de poder elegir un nuevo método de ordenamiento o finalizar la ejecución del programa.

➤ Manejo de Errores

Para el manejo de errores se revisó cada entrada incorrecta que pueda ingresar el usuario, así como un numero fuera del rango de elección o un carácter invalido no entero.

A sí mismo, el programa responde con un error si el archivo que lee, no está completamente llenado con enteros. En algún caso que se encuentre un String o carácter especial, el programa avisara al usuario de los errores, pasara el entero 0 y seguirá construyendo el arreglo de enteros.

En caso de que el archivo no este identificado con el nombre correcto o no se encuentre en la carpeta correspondiente, también nos dará un error y se visualizara un detalle del problema correspondiente al archivo.

Conclusion

En este trabajo se pudo entender y manejar el lenguaje de programación Java como así también el IDE Eclipse. Es una buena práctica para ejercitar el uso de paquetes y clases, como así también interiorizarse con el lenguaje, el cual tiene una inmensa utilización en diferentes sectores, ya que es un lenguaje muy popular entre los desarrolladores.

Sirve mucho realizar estos ejercicios para familiarizarse con el lenguaje, ya que es bastante intuitivo a la hora de programar. A sí mismo, el IDE Eclipse ofrece varias ayudas y un interfaz amigable para poder realizar de manera más eficiente el trabajo.

